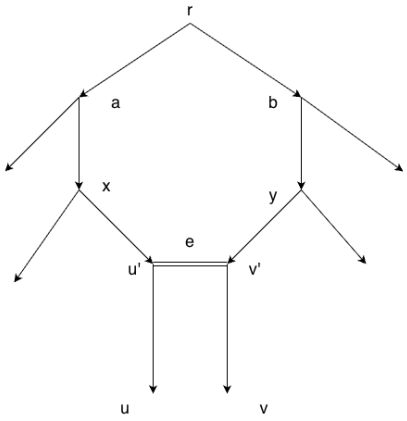


Figure 1: A directed network $\overline{N}$ rooted at r with a bidirectional edge e connecting u' and v' above u and v.

6 Extending CAMUS to bidirectional transfer edges

To extend CAMUS to work with bidirectional transfer edges, we need to modify Lemma 1 of the CAMUS paper [5], which states (reproduced below). Note that *all* trees are binary trees.

Lemma 1. Given a binary tree T and extension of T into level-1 phylogenetic network N, if quartet tree q is satisfied by N, then there is a unique non-tree edge e such that q is satisfied by $T + e$. Thus, the set of quartets that are satisfied by N is the disjoint union of quartets satisfied by $T + e$, as we let e vary among the non-tree edges of N.

This lemma itself follows from Observation 2 of Warnow *et al.* [4] (also reproduced below):

Observation 2. For every level-1 phylogenetic network N, $ab|cd \in Q(N_r)$ if and only if a, b is separated from c, d as the result of deleting either a cut edge in N or deleting two edges from a cycle, where one of the edges is incident with the bottom node of the cycle.

In our case, we have a rooted tree T and we would like to add *unrooted* edges to form this quasi-directed network N. To make absolutely clear that we are talking about rooted networks of this type, I propose the notation $\overline{N}$ to denote such a network (the bar reminiscent of the bidirectional lateral edges added to the underlying tree).

Please see Figure 1. Instead of placing the hybrid node at the bottom, place the hybridisation edge at the bottom. Then we can derive a similar observation to Observation 2 above (though I have not formally proved this):

Conjecture 1. *For every level-1 quasi-directed phylogenetic network $\overline{N}$, $ab|cd \in Q(\overline{N})$ if and only if a, b is separated from c, d as the result of deleting either a cut edge in $\overline{N}$ or deleting two edges from a cycle, where one of the edges is incident to the bottom edge of the cycle (in Figure 1, that would be either edge (x, u') or edge (y, v')).*

Proof. Proof sketch. Suppose the bottom edge is e, connecting nodes u' and v', as in Figure 1. A bidirectional edge e connecting x to y can be thought of as equivalent to two unidirectional edges, one from x to y and the other from y to x. If you first take the edge from x to y, y can be thought of as the "bottom node" in Observation 2, and in that case we must delete either a tree edge or an edge incident to y, which is either (y, v') or the extra edge e itself. WLOG we delete (y, v') because if we delete e then we are left with the original tree T. Ditto for the reverse case, where we either delete a tree edge, e, or (x, u'). Since e, (y, v'), and (x, u') are all incident to either u or v, Conjecture 1 holds. □

If Conjecture 1 holds, then I believe Lemma 1 also holds for extra edges e where e is bidirectional. If this holds, then the fundamental approach of CAMUS is sound, and modifying the CAMUS algorithm to support bidirectional transfer edges is simply a small redefinition of $\delta_T(q; u, w)$ to account for bidirectional edges, and then we just need to run the CAMUS dynamic programming algorithm.

I think the more interesting question is if CAMUS can be adapted to find lateral edges given *unrooted* tree topologies, e.g., those output from ASTRAL.

6.1 Generalising CAMUS to undirected trees

Conjecture 2. *Given a undirected tree T and an undirected level-1 network N that is an extension of T, if q is a quartet on the leafset of T that is not satisfied by T but* is *satisfied by N, then there is a unique edge $e \in E(N) \backslash E(T)$ such that q is satisfied by $T + e$.*

To prove the conjecture we first need the following lemma.

Lemma 6.0.1. *Suppose T is a binary tree, and $q = uv|xy$ a quartet. Suppose that the restriction of T to u, v, x, y is the topology $ux|vy$ (that is, T does not display q). Let $P = \overline{uv} \cap \overline{xy}$, that is, the overlapping route between $\overline{uv}$ and $\overline{xy}$. If N is a level-1 network that displays q, then all edges in P are in the same cycle.*

Proof. Suppose some edge in P was not in a cycle. Then it would be a cut edge separating u, x and v, y and thus all paths $u \to v$ and $x \to y$ must go through that edge. Then N would not display q, a contradiction.

Since we're working with level-1 networks, if we have two adjacent edges in P, then they must be in the same cycle (otherwise their shared endpoint would be in two cycles, breaking level-1). Hence there is one cycle that contains the entirety of P. □

Now we prove the above conjecture:

Proof. By the above lemma, there must be some edge e such that e is in the cycle with P - that edge is a viable candidate. Now suppose there is another edge $e' \neq e$ with the same claim: that $T + e'$ displays q. As N is level-1, the cycle that e' forms cannot contain P. Hence by the above lemma $T + e'$ cannot satisfy q, so e is the only viable candidate.

We now need to show that $T + e$ satisfies q, as otherwise we leave open the possibility of N displaying q but no single edge e exists such that $T + e$ displays q. That is, our assumption of N displays q implies that $T + e$ displays q. We omit the proof here, as it can be checked for all possible induced quarnets of N on u, v, x, y that this is true. □

Importantly, this proof holds regardless of the directedness of the underlying tree T or that of the extra edges e (though small modifications may have to be made to the proof). If conjecture 2 holds, then our DP formulation will still work. In particular, it still works even if we arbitrarily root the tree. Note that this works for both the completely undirected case as shown above and also the *semi-directed* case (where T is undirected but edges are directed), as nothing in the proof above relies on the directionality of the tree edges or the non-tree edges but merely tree topology.

Importantly, I think the this means that the general CAMUS algorithm works for adding edges to trees regardless if your starting tree is directed or if your extra tree edges are directed (one only has to modify the $\delta_T(q, (u, w))$ scoring function).

6.2 Improving CAMUS' computational complexity

I also have a few thoughts around the algorithmic complexity of CAMUS:

Conjecture 3. *It is possible to speed up CAMUS if we calculate $s_e(u, w)$ without explicitly calculating $\delta_T(q, (u, w))$ for all $q \in \mathcal{Q}$. This would bring down the bottleneck computational complexity of the $O(n^2|\mathcal{Q}|)$ term. Just a guess but I believe something like $\tilde{O}(\mathcal{Q} + n^2)$ is possible with the right preprocessing.*

Additionally, if we keep prefix sums in the dynamic programming phase I believe we can get down to $O(n^2)$ instead of $O(n^3)$ by computing s_{dp} in $O(n^2)$ time instead of $O(n^3)$ time.

The first part: we compute $s_e(u, v)$ directly. At a high level, for a given quartet q, we'd like to compute its contribution to all $s_e(u, v)$. We would like to increment $s(u, v)$ for which if you add an edge between the ancestor edges of u and v then q would be displayed. It turns out we just want to specify some constant number of pairs of paths P_1, P_2 on the tree (the exact paths depend on if the tree/edges are directed), and increment $s_e(u, v)$ for all $u, v \in P_1 \times P_2$. Naively this takes $O(n^2)$ time per quartet (and this is what CAMUS currently does), but in the following paragraph I will show how this can be sped up to $O(\log^2 n)$ per quartet using heavy-light decomposition.

Heavy-light decomposition is a technique where it is possible to "flattens" up a tree T with some nice locality properties. In particular, we can assign each node $x \in V(T)$ an index $1 \leq i(x) \leq n$ such that $y \neq x \iff i(y) \neq i(x)$ and for any two nodes $u, v \in V(T)$, the tree path $\overline{uv}$ can be split into $O(\log n)$ disjoint contiguous segments on the array. Hence two paths can be decomposed into $O(\log^2 n)$ rectangles. Then for a given pair of paths, incrementing all pairs of nodes with one on each path becomes a 2D increment of $O(\log^2 n)$ rectangles. Adding to a rectangle takes constant time as we are able to do this offline and can maintain some sort of difference array to take the prefix sum of later. Preprocessing LCAs and then computing s_e now takes $O(n^2 + |\mathcal{Q}| \log^2 n)$ time.

With both optimisations, the computational complexity of CAMUS can be reduced down to $O\left(n^2 + |\mathcal{Q}| \log^2 n\right)$.

References

[1] Marc E Canby et al. "Addressing polymorphism in linguistic phylogenetics". In: *Transactions of the Philological Society* 122.2 (2024), pp. 191–222.

[2] Yunheng Han and Erin K Molloy. "Improved robustness to gene tree incompleteness, estimation errors, and systematic homology errors with weighted TREE-QMC". In: *Systematic Biology* 74.5 (2025), pp. 758–777.

[3] Cuong Than, Derek Ruths, and Luay Nakhleh. "PhyloNet: a software package for analyzing and reconstructing reticulate evolutionary relationships". In: *BMC bioinformatics* 9.1 (2008), p. 322.

[4] Tandy Warnow, Yasamin Tabatabaee, and Steven N Evans. "Advances in estimating level-1 phylogenetic networks from unrooted SNPs". In: *Journal of Computational Biology* 32.1 (2025), pp. 3–27.

[5] James Willson and Tandy Warnow. "CAMUS: Scalable Phylogenetic Network Estimation". In: *bioRxiv* (2026), pp. 2026–02.

[6] Chao Zhang, Rasmus Nielsen, and Siavash Mirarab. "ASTER: a package for large-scale phylogenomic reconstructions". In: *Molecular Biology and Evolution* 42.8 (2025), msaf172.

[7] Chao Zhang et al. "ASTRAL-III: polynomial time species tree reconstruction from partially resolved gene trees". In: *BMC bioinformatics* 19.Suppl 6 (2018), p. 153.